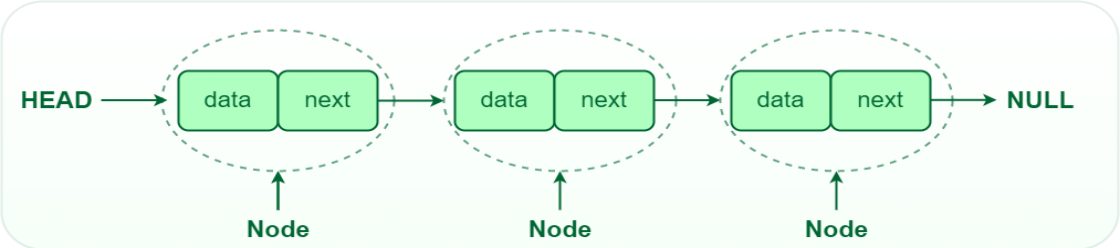
UNIT-III

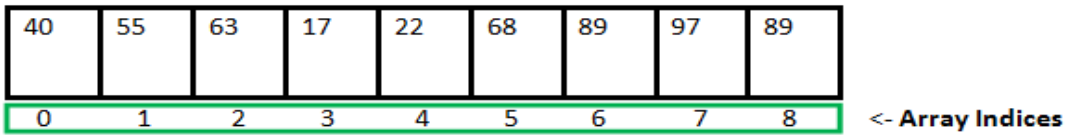
Linked Lists: Singly linked lists: representation and operations, doubly linked lists and its operations and circular linked lists and its operations, Comparing arrays and linked lists, Applications of linked lists.

Linked List:

Linked List is a linear data structure, in which elements are not stored at a contiguous location, rather they are linked using pointers. Linked List forms a series of connected nodes, where each node stores the data and the address of the next node.

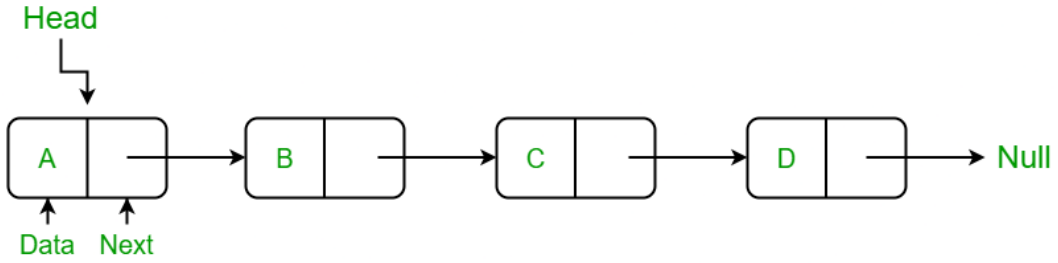


Array: Arrays store elements in contiguous memory locations, resulting in easily calculable addresses for the elements stored and this allows faster access to an element at a specific index.



Array Length = 9
First Index = 0
Last Index = 8

Linked List: Linked lists are less rigid in their storage structure and elements are usually not stored in contiguous locations, hence they need to be stored with additional tags giving a reference to the next element.



Advantages of Linked List over arrays :

- **Efficient insertion and deletion**: Linked lists allow insertion and deletion in the middle in O(1) time, if we have a pointer to the target position, as only a few pointer changes are needed. In contrast, arrays require O(n) time for insertion or deletion in the middle due to element shifting.

- **Implementation of Queue and Deque** : Simple array implementation is not efficient at all. We must use circular array to efficiently implement which is complex. But with linked list, it is easy and straightforward. That is why most of the language libraries use Linked List internally to implement these data structures.
- **Space Efficient in Some Cases :** Linked List might turn out to be more space efficient compare to arrays in cases where we cannot guess the number of elements in advance. In case of arrays, the whole memory for items is allocated together. Even with dynamic sized arrays like vector in C++ or list in Python or ArrayList in Java. the internal working involves de-allocation of whole memory and allocation of a bigger chunk when insertions happen beyond the current capacity.
- **Circular List with Deletion/Addition :** Circular Linked Lists are useful to implement CPU round robin scheduling or similar requirements in the real world because of the quick deletion/insertion in a circular manner.

Advantages of Arrays over Linked List :

- **Random Access**. : We can access ith item in O(1) time (only some basic arithmetic required using base address). In case of linked lists, it is O(n) operation due to sequential access**.**
- **Cache Friendliness** : Array items (Or item references) are stored at contiguous locations which makes array cache friendly (Please refer Spatial locality of reference for more details)
- **Easy to use :** Arrays are relatively very easy to use and are available as core of programming languages
- **Less Overhead :** Unlike linked list, we do not have any extra references / pointers to be stored with every item.

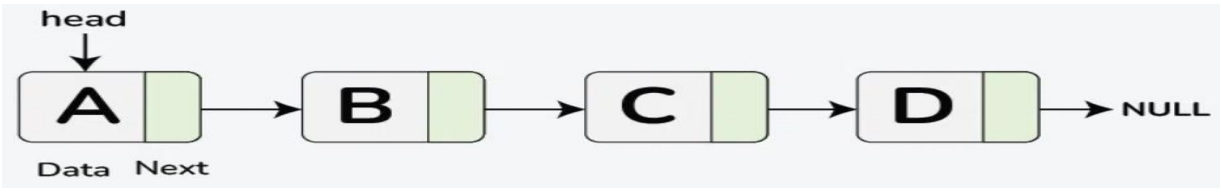
Types Of Linked Lists:

1. **Singly Linked List**
2. **Doubly Linked List**
3. **Circular Linked List**

Singly Linked List:

A singly linked list is a fundamental data structure, it consists of nodes where each node contains a data field and a reference to the next node in the linked list. The next of the last node is null, indicating the end of the list. Linked Lists support efficient insertion and deletion operations.

In a singly linked list, each node consists of two parts: data and a pointer to the next node. This structure allows nodes to be dynamically linked together, forming a chain-like sequence.

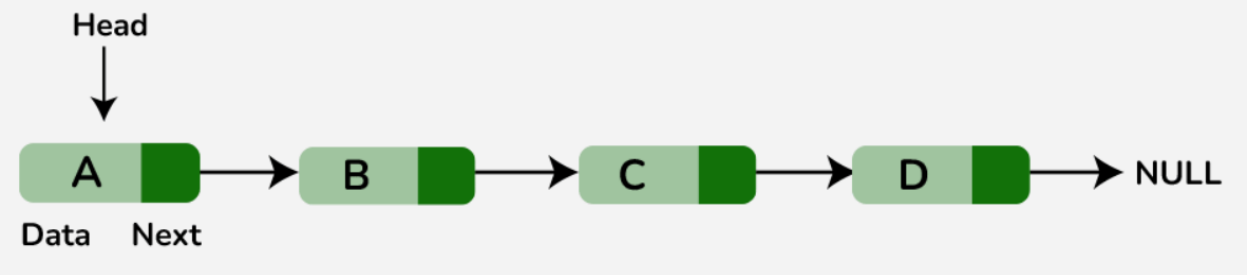


```
// Definition of a Node in a singly linked list
struct Node {
    int data;
    struct Node* next;
};
// Function to create a new Node
struct Node* newNode(int data) {
    struct Node* temp =
      (struct Node*)malloc(sizeof(struct Node));
    temp->data = data;
    temp->next = NULL;
    return temp;
}
```

In this example, the Node structure contains an integer data field (**data**) to store the information and a pointer to another Node (**next**) to establish the link to the next node in the list.

Traversal of Singly Linked List:
Traversal in a linked list means visiting each node and performing operations like printing or processing data.
Step-by-step approach:

1. Initialize a pointer (current) to the head of the list.
2. Loop through the list using a while loop until current becomes NULL.
3. Process each node (e.g., print its data).
4. Move to the next node by updating current = current->next.



Examples:
Input: 1->2->3->4->5->null
Output: 1 2 3 4 5
Explanation: Every element of each node from head node to last node is printed which means we have traversed each node successfully.
Input: 10->20->30->40->50->null
Output: 10 20 30 40 50
Explanation: Every element of each node from head node to last node is printed which means we have traversed each node successfully.
Input: 5->10->15->20->25->null
Output: 5 10 15 20 25

Explanation: Each node's value is printed sequentially from the head to the last node, confirming successful traversal.

Searching in Singly Linked List:
Searching in a Singly Linked List refers to the process of looking for a specific element or value within the elements of the linked list.
Step-by-step approach:

1. **Start from the head** of the linked list.
2. **Check each node's data**:
    - If it matches the target value, **r**eturn true (element found).
    - Otherwise, move to the next node.
3. **Repeat until** the end (NULL) is reached.
4. If no match is found, **return false**.

Given a linked list and a **key**, the task is to check if **key** is present in the linked list or not.
Examples:
Input: 14 -> 21 -> 11 -> 30 -> 10, key = 14
Output: Yes
Explanation: 14 is present in the linked list.
Input: 6 -> 21 -> 17 -> 30 -> 10 -> 8, key = 13
Output: No
Explanation: No node in the linked list has value = 13.
Input: 9 -> 18 -> 27 -> 36 -> 45, key = 27
Output: Yes
Explanation: 27 is present in the linked list.

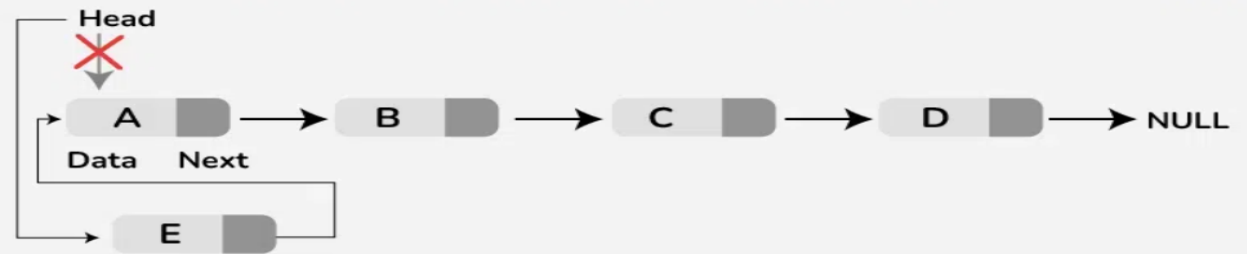
Length of Singly Linked List:
Finding the length of a **Singly Linked List** means counting the total number of nodes.
Step-by-step approach:

1. **Initialize** a counter (length = 0).
2. **Start from the head**, assign it to current.
3. **Traverse the list**:
    - **Increment length** for each node.
    - **Move to the next node** (current = current->next).
4. **Return the final length** when current becomes NULL.

Given a Singly Linked List, the task is to find the Length of the Linked List.
Examples:
Input: LinkedList = 1->3->1->2->1
Output: 5
Explanation: The linked list has 5 nodes.
Input: LinkedList = 2->4->1->9->5->3->6
Output: 7
Explanation: The linked list has 7 nodes.

Input: LinkedList = 10->20->30->40->50->60
Output: 6
Explanation: The linked list has 6 nodes.

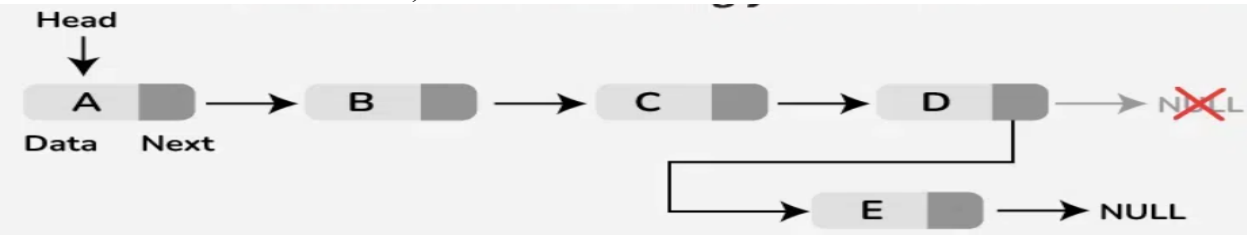
Insertion in Singly Linked List:
Insertion is a fundamental operation in linked lists that involves adding a new node to the list. There are several scenarios for insertion:
Insertion at the Beginning of Singly Linked List: Insertion at the beginning involves adding a new node before the current head, making it the new head.



Step-by-step approach:

- Create a new node with the given value.
- Set the **next** pointer of the new node to the current head.
- Move the head to point to the new node.
- Return the new head of the linked list.

Insertion at the End of Singly Linked List: To insert a node at the end of the list, traverse the list until the last node is reached, and then link the new node to the current last node



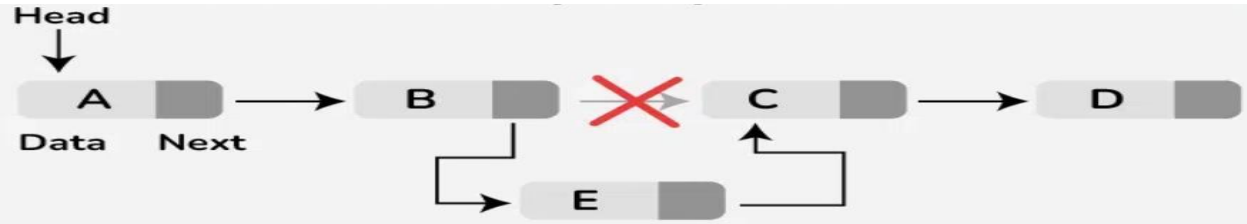
Step-by-step approach:

- Create a new node with the given value.
- Check if the list is empty:
  - If it is, make the new node the head and return.
- Traverse the list until the last node is reached.
- Link the new node to the current last node by setting the last node's next pointer to the new node.

Insertion at a Specific Position of the Singly Linked List: To insert a node at a specific position, traverse the list to the desired position, link the new node to the next node, and update the links accordingly.
Step-by-step approach:

- **Create a new node** and assign it a value.
- **If inserting at the beginning** (position = 1):
  - Point the new node's next to the current head.
  - Update the head to the new node.

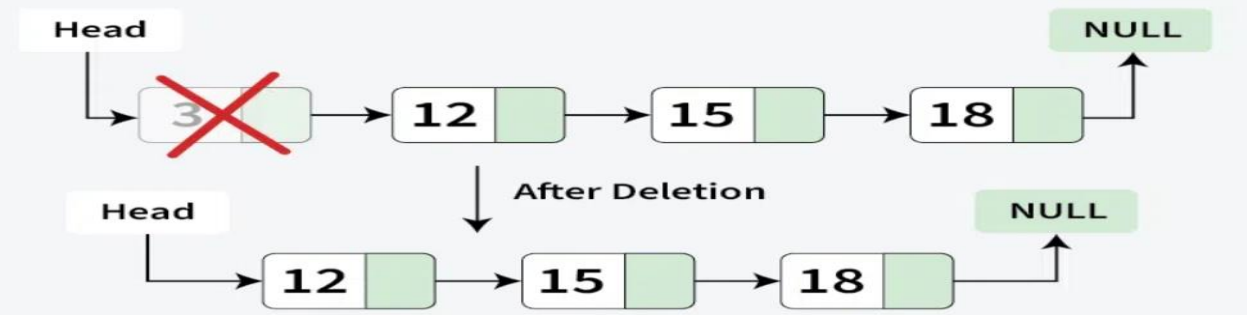
- **Return** (Insertion done).
- **Otherwise, traverse the list**:
  - Start from the head and move to the (position - 1)th node (just before the desired position).
  - If the position is beyond the list length, **return an error or append at the end**.
- **Insert the new node**:
  - Point the new node's next to the next node of the current position.
  - Update the previous node's next to the new node.
- **Return the updated list.**



Deletion in Singly Linked List

Deletion involves removing a node from the linked list. Similar to insertion, there are different scenarios for deletion:

Deletion at the Beginning of Singly Linked List: To delete the first node, update the head to point to the second node in the list.



Steps-by-step approach:

- Check if the head is **NULL**.
  - If it is, return **NULL** (the list is empty).
- Store the current head node in a temporary variable **temp**.
- Move the head pointer to the next node.
- Delete the temporary node.
- Return the new head of the linked list.

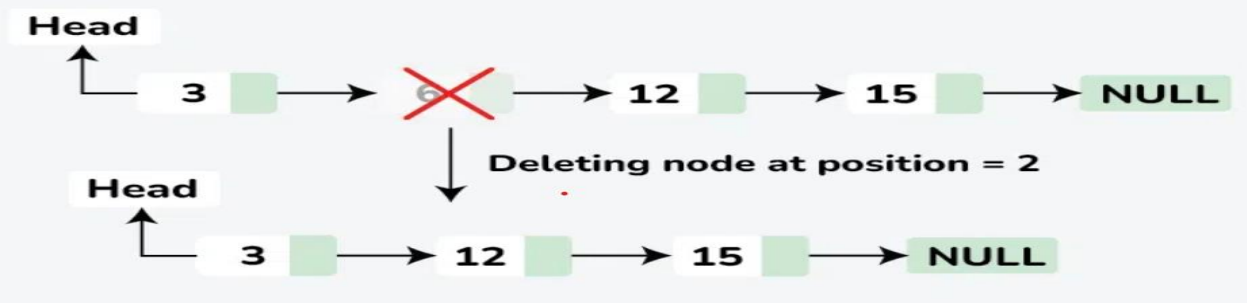
Deletion at the End of Singly Linked List: To delete the last node, traverse the list until the second-to-last node and update its next field to None.



Step-by-step approach:

- Check if the head is **NULL**.
  - If it is, return NULL (the list is empty).
- Check if the head's **next** is **NULL** (only one node in the list).
  - If true, delete the head and return **NULL**.
- Traverse the list to find the second last node (**second_last**).
- Delete the last node (the node after **second_last**).
- Set the **next** pointer of the second last node to **NULL**.
- Return the head of the linked list.

Deletion at a Specific Position of Singly Linked List: To delete a node at a specific position, traverse the list to the desired position, update the links to bypass the node to be deleted.



Step-by-step approach:

- Check if the list is empty or the position is invalid, return if so.
- If the head needs to be deleted, update the head and delete the node.
- Traverse to the node before the position to be deleted.
- If the position is out of range, return.
- Store the node to be deleted.
- Update the links to bypass the node.
- Delete the stored node.

Program:

```
//Write a C Program to implement single linked list operations
#include<stdio.h>
#include<stdlib.h>
struct node
{
        int data;
        struct node *next;
};
typedef struct node NODE;
NODE *create();
void insertAtBegin(int);
void insertAtEnd(int);
void insertAtAnywhere(int);
void deleteAtBegin();
void deleteAtEnd();
void deleteAtAnywhere();
```

```
void traverse();
void reverse(NODE*);
int size();
NODE *head=NULL;
NODE *create()
{
	NODE *temp=(NODE*)malloc(sizeof(NODE));
	return temp;
}
void insertAtBegin(int n)
{
	NODE *temp=create();
	temp->data=n;
	if(head==NULL){
		head=temp;
		head->next=NULL;
	}
	else
	{
		temp->next=head;
		head=temp;
	}
}
void insertAtEnd(int n)
{
	NODE *t,*temp=create();
	temp->data=n;
	temp->next=NULL;
	if(head==NULL)
	{
		head=temp;
	}
	else
	{
		t=head;
		while(t->next!=NULL)
		t=t->next;
		t->next=temp;
	}
}
void insertAtAnywhere(int n)
{
	NODE *t,*prev,*temp;
	int p,loc=0;
	printf("Enter position to insert:");
	scanf("%d",&p);
```

```
        if(head==NULL)
        {
                printf("No such postion:\n");
        }
        else if(p==size()+1)
        insertAtEnd(n);
        else if(p>size())
        printf("No Such position\n");
        else
        {
                temp=create();
                temp->data=n;
                t=prev=head;
                while(t->next!=NULL)
                {
                        loc++;
                        if(loc==p)
                        break;
                        prev=t;
                        t=t->next;
                }
                prev->next=temp;
                temp->next=t;

        }
}
int size()
{
        NODE *temp=head;
        int l=0;
        while(temp!=NULL)
        {
                l++;
                temp=temp->next;
        }
        return l;
}
void deleteAtBegin()
{
        NODE *temp;
        if(head==NULL)
        {
                printf("List is empty!!!\n");
                return;
        }
        else
```

```
	{
		printf("%d is removed\n",head->data);
		temp=head;
		head=head->next;
		free(temp);
	}
}
void deleteAtEnd()
{
	NODE *temp,*prev;
	if(head==NULL)
	{
		printf("List is empty!!!\n");
		return;
	}
	else
	{
		temp=head;
		while(temp->next!=NULL)
		{
			prev=temp;
			temp=temp->next;
		}
		printf("%d is removed\n",temp->data);
		prev->next=NULL;
		free(temp);
	}
}
void deleteAtAnywhere()
{
	NODE *temp,*prev;
	int p,loc=0;
	printf("Enter the location:");
	scanf("%d",&p);
	if(head==NULL)
	{
		printf("List is Empty!!!\n");
	}
	else if(p==size())
	deleteAtEnd();
	else if(p==1)
	deleteAtBegin();
	else if(p>size())
	printf("No such position\n");
	else
	{
```

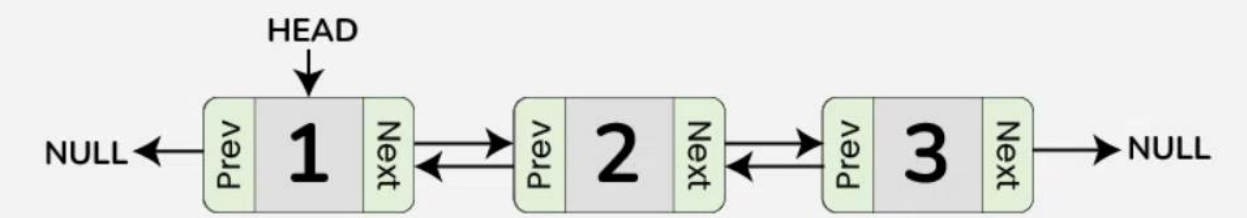
```
                temp=head;
                while(temp->next!=NULL)
                {
                        loc++;
                        if(loc==p)
                        break;
                        prev=temp;
                        temp=temp->next;
                }
                printf("%d is removed\n",temp->data);
                prev->next=temp->next;
                free(temp);
        }
}
void reverse(NODE *t)
{
        if(t==NULL)
        return;
        else
        {
                reverse(t->next);
                printf("->%d",t->data);
        }
}
void traverse()
{
        NODE *temp;
        if(head==NULL)
        printf("List is empty!!!\n");
        else
        {
                temp=head;
                while(temp!=NULL)
                {
                        printf("->%d",temp->data);
                        temp=temp->next;
                }
                printf("\n");
        }
}
int main()
{
        int ch,n;
        do
        {
                printf("1.insert at begin\n2.Insert at end\n");
```

```
                printf("3.insert at anywhere\n4.Delete at begin\n");
                printf("5.delete at ending\n6.delete at anywhere\n");
                printf("7.traverse\n8.reverse\nEnter your choice:");
                scanf("%d",&ch);
                switch(ch)
                {
                        case 1:printf("Enter an Element to insert:");
                                scanf("%d",&n);
                                insertAtBegin(n);
                                break;
                        case 2:printf("Enter an Element to insert:");
                                scanf("%d",&n);
                                insertAtEnd(n);
                                break;
                        case 3:printf("Enter an Element to insert:");
                                scanf("%d",&n);
                                insertAtAnywhere(n);
                                break;
                        case 4:deleteAtBegin();
                                break;
                        case 5:deleteAtEnd();
                                break;
                        case 6:deleteAtAnywhere();
                                break;
                        case 7:traverse();
                                break;
                        case 8:if(head==NULL)
                                printf("List is Empty!!!");
                                else
                                reverse(head);
                                printf("\n");
                                break;
                        default:exit(0);
                }
        }while(1);
        return 0;
}
```

Output:	1.insert at begin	1.insert at begin
1.insert at begin	2.Insert at end	2.Insert at end
2.Insert at end	3.insert at anywhere	3.insert at anywhere
3.insert at anywhere	4.Delete at begin	4.Delete at begin
4.Delete at begin	5.delete at ending	5.delete at ending
5.delete at ending	6.delete at anywhere	6.delete at anywhere
6.delete at anywhere	7.traverse	7.traverse
7.traverse	8.reverse	8.reverse
8.reverse	Enter your choice:3	Enter your choice:5

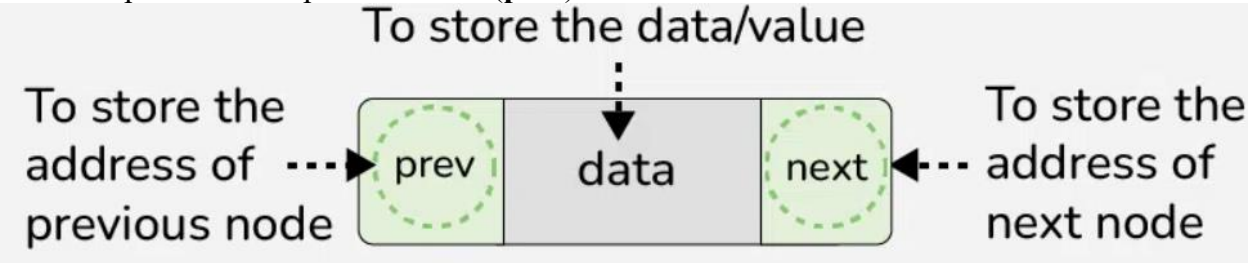
Enter your choice:1 Enter an Element to insert:10 1.insert at begin 2.Insert at end 3.insert at anywhere 4.Delete at begin 5.delete at ending 6.delete at anywhere 7.traverse 8.reverse Enter your choice:2 Enter an Element to insert:20 1.insert at begin 2.Insert at end 3.insert at anywhere 4.Delete at begin 5.delete at ending 6.delete at anywhere 7.traverse 8.reverse Enter your choice:2 Enter an Element to insert:30 1.insert at begin 2.Insert at end 3.insert at anywhere 4.Delete at begin 5.delete at ending 6.delete at anywhere 7.traverse 8.reverse Enter your choice:2 Enter an Element to insert:40 1.insert at begin 2.Insert at end 3.insert at anywhere 4.Delete at begin 5.delete at ending 6.delete at anywhere 7.traverse 8.reverse Enter your choice:7 ->10->20->30->40	Enter an Element to insert:50 Enter position to insert:3 1.insert at begin 2.Insert at end 3.insert at anywhere 4.Delete at begin 5.delete at ending 6.delete at anywhere 7.traverse 8.reverse Enter your choice:7 ->10->20->50->30->40 1.insert at begin 2.Insert at end 3.insert at anywhere 4.Delete at begin 5.delete at ending 6.delete at anywhere 7.traverse 8.reverse Enter your choice:8 ->40->30->50->20->10 1.insert at begin 2.Insert at end 3.insert at anywhere 4.Delete at begin 5.delete at ending 6.delete at anywhere 7.traverse 8.reverse Enter your choice:4 10 is removed 1.insert at begin 2.Insert at end 3.insert at anywhere 4.Delete at begin 5.delete at ending 6.delete at anywhere 7.traverse 8.reverse Enter your choice:7 ->20->50->30->40	40 is removed 1.insert at begin 2.Insert at end 3.insert at anywhere 4.Delete at begin 5.delete at ending 6.delete at anywhere 7.traverse 8.reverse Enter your choice:7 ->20->50->30 1.insert at begin 2.Insert at end 3.insert at anywhere 4.Delete at begin 5.delete at ending 6.delete at anywhere 7.traverse 8.reverse Enter your choice:6 Enter the location:2 50 is removed 1.insert at begin 2.Insert at end 3.insert at anywhere 4.Delete at begin 5.delete at ending 6.delete at anywhere 7.traverse 8.reverse Enter your choice:7 ->20->30 1.insert at begin 2.Insert at end 3.insert at anywhere 4.Delete at begin 5.delete at ending 6.delete at anywhere 7.traverse 8.reverse Enter your choice:9

Doubly Linked List:
A doubly linked list is a more complex data structure than a singly linked list, but it offers several advantages. The main advantage of a doubly linked list is that it allows for efficient traversal of the list in both directions. This is because each node in the list contains a pointer to the previous node and a pointer to the next node. This allows for quick and easy insertion and deletion of nodes from the list, as well as efficient traversal of the list in both directions.



Representation of Doubly Linked List in Data Structure
In a data structure, a doubly linked list is represented using nodes that have three fields:

1. Data
2. A pointer to the next node (**next**)
3. A pointer to the previous node (**prev**)



Node Definition
Here is how a node in a Doubly Linked List is typically represented:

```
struct Node {
    // To store the Value or data.
    int data;
    // Pointer to point the Previous Element
    Node* prev;
    // Pointer to point the Next Element
    Node* next;
};
// Function to create a new node
struct Node *createNode(int new_data) {
    struct Node *new_node = (struct Node *)
    malloc(sizeof(struct Node));
    new_node->data = new_data;
    new_node->next = NULL;
    new_node->prev = NULL;
    return new_node;
}
```

Basic Operations on C Doubly Linked List

We can perform the following basic operations in a doubly-linked list:

- Insertion
- Deletion
- Traversal

1. Insertion in Doubly Linked List in C

Inserting a new node in a doubly linked list can be done in a similar way like inserting a new node in a singly linked list but we have to maintain the link of previous node also. We have three scenarios while inserting a node in a doubly linked list:

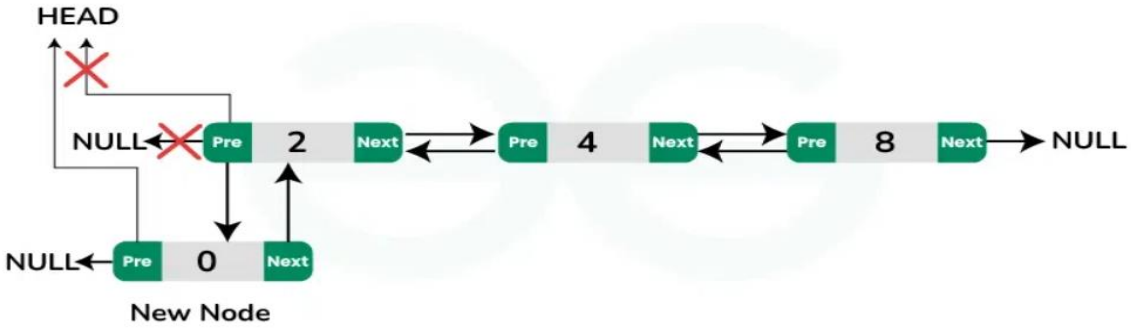
- Insertion at the beginning of the list.
- Insertion at the end of the list.
- Insertion in the middle of the list.

a. Insertion at the Beginning of the Doubly Linked List

To insert a new node at the beginning of the doubly linked list follow the below approach:

Approach:

- Create a newNode with **data** field assigned the given value.
- If the list is empty:
  - Set newNode->next to NULL.
  - Set newNode->prev to NULL.
  - Update the head pointer to point to newNode.
- If the list is not empty:
  - Set newNode->next to head.
  - Set newNode->prev to NULL.
  - Set head->prev to newNode.
  - Update the head pointer to point to newNode.



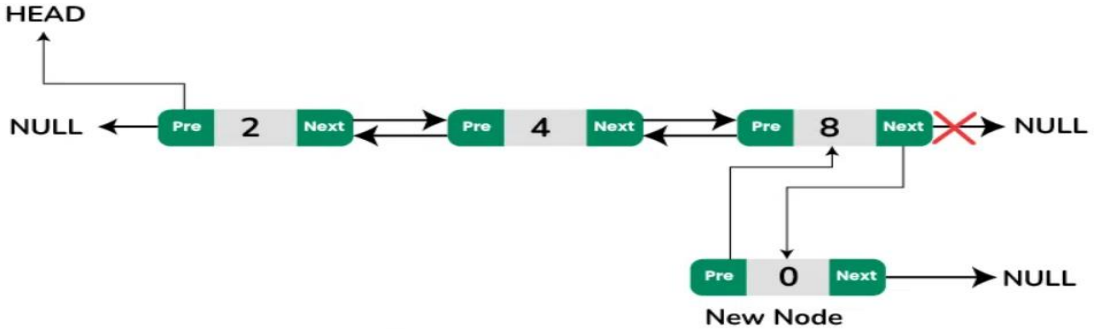
b. Insertion at the End of the Doubly Linked List

To insert a new node at the end of the doubly linked list, follow the below approach:

Approach:

- Create a newNode with **data** field assigned the given value.
- If the list is empty:
  - Set newNode->next to NULL.
  - Set newNode->prev to NULL.
  - Update the head pointer to point to newNode.
- If the list is not empty:
  - Traverse the list to find the last node (where last->next == NULL).
  - Set last->next to newNode.
  - Set newNode->prev to last.

- Set newNode->next to NULL.

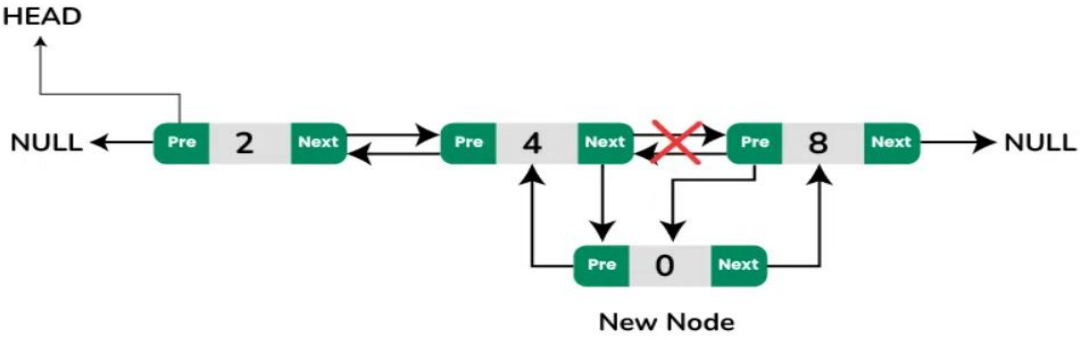


c. Insertion in the Middle of the Doubly Linked List

For inserting a node at a specific position in a doubly linked list follow the below approach:

Approach:

- *Create a newNode.*
- *Find the size of the list to ensure the position is within valid bounds.*
- *If the position is 0 (or 1 depending on your indexing):*
  - *Use the insertion at the beginning approach.*
- *If the position is equal to the size of the list:*
  - *Use the insertion at the end approach.*
- *If the position is valid and not at the boundaries:*
  - *Traverse the list to find the node immediately before the desired insertion point (prevNode).*
  - *Set newNode->next to prevNode->next.*
  - *Set newNode->prev to prevNode.*
  - *If prevNode->next is not NULL, set prevNode->next->prev to newNode.*
  - *Set prevNode->next to newNode.*



2. Deletion in a Doubly Linked List in C

Just like insertion, we have three scenarios while deleting a node in a doubly linked list:

- Deletion from the beginning of the list.
- Deletion from the end of the list.
- Deletion at specific position of the list.

a. Deletion at the Beginning of the Doubly Linked List

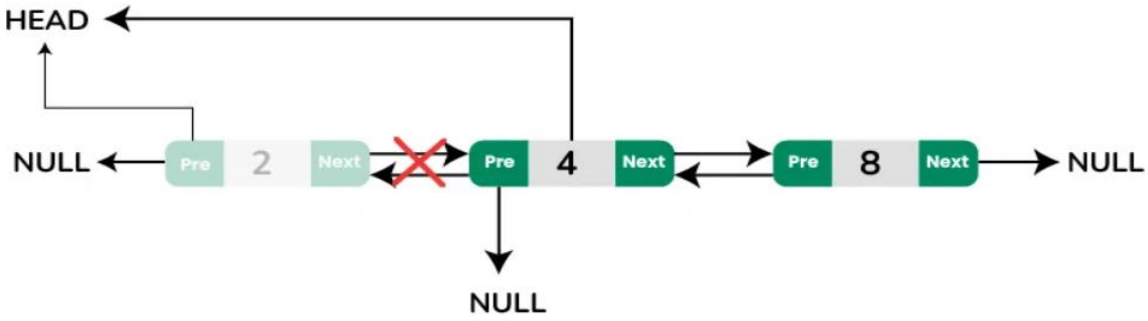
To delete a node at the beginning of the doubly linked list, follow the below approach:

Approach:

- ***Check if the List is Empty****: If true, there's nothing to delete.*
- ***Check if the List Contains Only One Node****:*
  - *Set head to NULL.*

- *Free the memory of the node.*

- ***If the List Contains More Than One Node**:*
  - *Update head to head->next.*
  - *Set the prev of the new head to NULL.*
  - *Free the memory of the old head.*

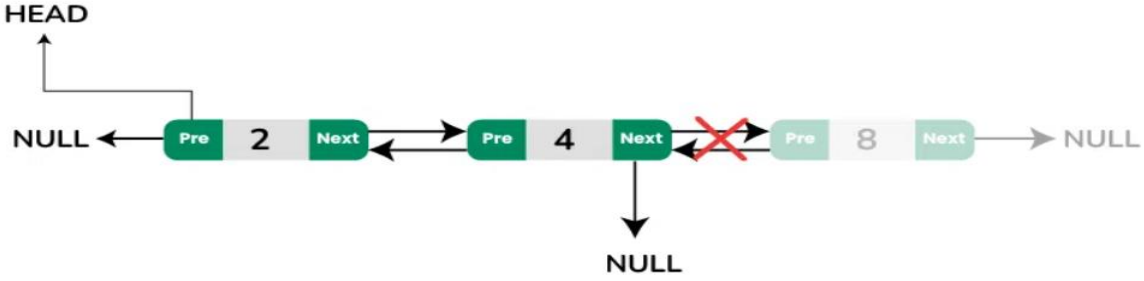


b. Deletion at the End of the Doubly Linked List

To delete a node at the end of the doubly linked list, follow the below approach:

Approach:

- *Check if the List is Empty: If true, there's nothing to delete.*
- *If the List is Not Empty:*
  - *Traverse to the last node using a loop.*
  - *Check if the last node is the only node (head->next == NULL):*
    - *Update head to NULL.*
  - *If more than one node:*
    - *Set the next of the second last node (last->prev) to NULL.*
  - *Free the memory of the last node.*



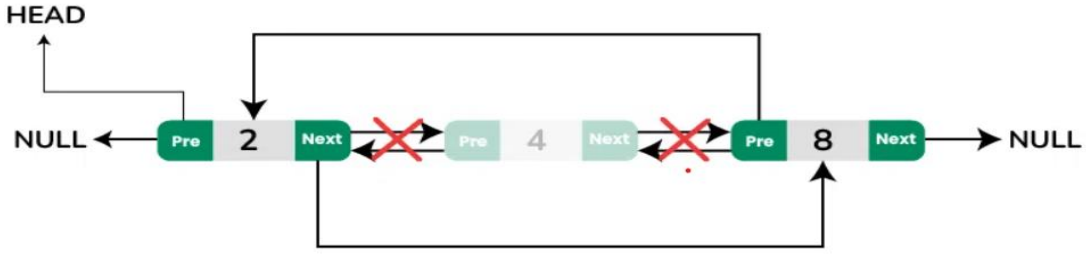
c. Deletion at a Specific Position in Doubly Linked List

For deleting a node at a specific position in a doubly linked list, follow the below approach:

Approach:

- ***Check Position Validity**:*
  - *If position is 0 (or 1 depending on indexing), use deletion at the beginning.*
  - *If position is the size of the list, use deletion at the end.*
  - *Otherwise, proceed with the middle deletion.*
- ***Traverse to the Specified Position**:*
  - *Use a loop to reach the node (delNode) at the desired position.*
  - *If delNode is the first node, adjust head.*
  - *If not, adjust delNode->prev->next and delNode->next->prev if delNode->next is not NULL.*
- ***Free the Memory**:*

- *Release the node to be deleted.*



3. Traversal in a Doubly Linked List in C

Traversal in a doubly linked list means visiting each node of the doubly linked list to perform some kind of operation like displaying the data stored in that node. Unlike singly linked list we can traverse in doubly linked list in both the directions.

- **Forward Traversal:** From head node to tail node
- **Reverse Traversal:** From tail node to head node

a. Forward Traversal in Doubly Linked List

For traversing in a forward direction in a doubly linked list, follow the below approach:

Approach:

- *Create a temporary pointer ptr and copy the head pointer into it.*
- *Traverse through the list using a while loop.*
- *Keep moving the value of temp pointer variable ptr to ptr->next until the last node whose next part contains null is found.*
- *During each iteration of the loop, perform the desired operation.*

b. Reverse Traversal in Doubly Linked List

For traversing in a reverse direction in a doubly linked list, follow the below approach:

- *Create a temporary pointer ptr and copy the tail pointer into it.*
- *Traverse through the list using a while loop.*
- *Keep moving the value of temp pointer variable ptr to ptr->prev until the first node whose prev part contains null is found.*
- *During each iteration of the loop, perform the desired operation .*

Program

```
//Write a C program to implement double linked list operation
#include<stdio.h>
#include<stdlib.h>
typedef struct node
{
        struct node *prev;
        int data;
        struct node *next;
} NODE;
NODE *head=NULL;
NODE *create();
void insertAtBegin();
void insertAtEnd();
void insertAtAnywhere();
void deleteAtBegin();
void deleteAtEnd();
```

```
void deleteAtAnywhere();
void traverse();
void reverse(NODE*);
int size=0;
NODE *create()
{
	NODE *temp=(NODE*)malloc(sizeof(NODE));
	printf("\nEnter data:");
	scanf("%d",&temp->data);
	temp->prev=NULL;
	temp->next=NULL;
	size++;
	return temp;
}
void insertAtBegin()
{
	NODE *temp=create();
	if(head==NULL)
	{
		head=temp;
	}
	else
	{
		temp->next=head;
		head=temp;
	}
}
void insertAtEnd()
{
	NODE *new=create(),*t;
	if(head==NULL)
	{
		head=new;
	}
	else
	{
		t=head;
		while(t->next!=NULL)
		t=t->next;
		t->next=new;
		new->prev=t;
	}
}
void insertAtAnywhere()
{
	int p,loc=0;
```

```
        printf("Enter position to insert:");
        scanf("%d",&p);
        if(head==NULL)
        printf("No Such postion\n");
        else if(p==1)
        insertAtBegin();
        else if(p==size+1)
        insertAtEnd();
        else if(p>size)
        printf("No Such postion\n");
        else
        {
                NODE *new,*t,*pr;
                new=create();
                t=pr=head;
                while(t->next!=NULL)
                {
                        loc++;
                        if(loc==p)
                        break;
                        pr=t;
                        t=t->next;
                }
                new->prev=t->prev;
                new->next=t;
                t->prev=new;
                pr->next=new;
        }
}
void deleteAtBegin()
{
        NODE *temp;
        if(head==NULL)
        {
                printf("List is Empty!!!\n");
        }
        else
        {
                temp=head;
                head=head->next;
                head->prev=NULL;
                size--;
                printf("%d is removed\n",temp->data);
                free(temp);
        }
}
```

```
void deleteAtEnd()
{
	NODE *temp=head;
	if(head==NULL)
	printf("List is empty!!!\n");
	else
	{
		while(temp->next!=NULL)
		temp=temp->next;
		temp->prev->next=NULL;
		size--;
		printf("%d is removed\n",temp->data);
		free(temp);
	}
}
void deleteAtAnywhere()
{
	NODE *temp,*t;
	int p,loc=0;
	if(head==NULL)
	printf("List is Empty!!!\n");
	else
	{
		printf("\nEnter the position to insert:");
		scanf("%d",&p);
		if(p==1)
		deleteAtBegin();
		else if(p==size)
		deleteAtEnd();
		else if(p>size)
		printf("No Such postion.\n");
		else
		{
			temp=t=head;
			while(temp->next!=NULL)
			{
				loc++;
				if(loc==p)
				break;
				t=temp;
				temp=temp->next;
			}
			temp->next->prev=t;
			t->next=temp->next;
			size--;
			printf("%d is removed\n",temp->data);
```

```
                        free(temp);
                }
        }
}
void reverse(NODE *temp)
{
        if(head==NULL)
        printf("List is Empty!!!\n");
        else if(temp==NULL)
        return;
        else
        {
                reverse(temp->next);
                printf("-->%d",temp->data);
        }
}
void traverse()
{
        NODE *t;
        if(head==NULL)
        printf("List is Empty.\n");
        else
        {
                t=head;
                while(t!=NULL)
                {
                        printf("<-->%d",t->data);
                        t=t->next;
                }
                printf("\n");
        }
}

int main()
{
        int ch;
        do
        {
                printf("1.Insert at beginning.\n");
                printf("2.Insert At Ending.\n");
                printf("3.Insert At Any where.\n");
                printf("4.Deletion at bigenning.\n");
                printf("5.Deletion at ending.\n");
                printf("6.Deletion At Anywhere.\n");
                printf("7.Traverse.\n");
                printf("8.Reverse.\n");
```

```
                printf("9.Count.\n");
                printf("Enter your choice:");
                scanf("%d",&ch);
                //printf("number of elements are %d\n",size);
                switch(ch)
                {
                        case 1:insertAtBegin();
                                break;
                        case 2:insertAtEnd();
                                break;
                        case 3:insertAtAnywhere();
                                break;
                        case 4:deleteAtBegin();
                                break;
                        case 5:deleteAtEnd();
                                break;
                        case 6:deleteAtAnywhere();
                                break;
                        case 7:traverse();
                                break;
                        case 8:reverse(head);
                                printf("\n");
                                break;
                        case 9:printf("Number of elements are %d\n",size);
                                break;
                        default:exit(0);
                }
        }while(1);
        return 0;
}
```

Output:

1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:1 Enter data:10 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning.	1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:3 Enter position to insert:2 Enter data:15 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where.	1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:5 30 is removed 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning.

5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:2 Enter data:20 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:2 Enter data:30 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:7 <-->10<-->20<-->30 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:3 Enter position to insert:2 Enter data:15 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending.	4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:7 <-->10<-->15<-->20<-->30 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:6 Enter the position to insert:2 15 is removed 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:7 <-->10<-->20<-->30 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:8 -->30-->20-->10 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning.	5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:7 <-->10<-->20 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:4 10 is removed 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:7 <-->20 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:10

6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:7 <-->10<-->20<-->30	5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Reverse. 9.Count. Enter your choice:9 Number of elements are 3	

Circular Linked List:

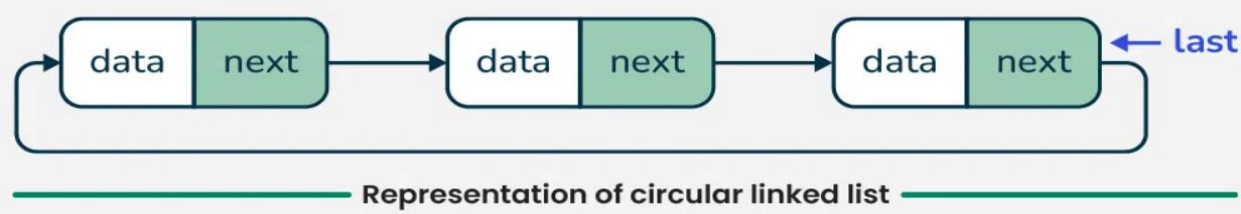
A circular linked list is a special type of linked list where all the nodes are connected to form a circle. Unlike a regular linked list, which ends with a node pointing to NULL, the last node in a circular linked list points back to the first node. This means that you can keep traversing the list without ever reaching a NULL value.

Types of Circular Linked Lists

We can create a circular linked list from both singly linked lists and doubly linked lists. So, circular linked lists are basically of two types:

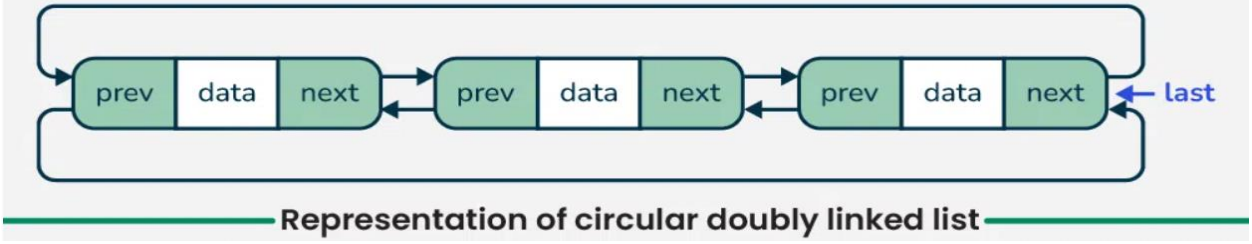
1. Circular Singly Linked List

In Circular Singly Linked List, each node has just one pointer called the "next" pointer. The next pointer of the last node points back to the first node and this results in forming a circle. In this type of Linked list, we can only move through the list in one direction.

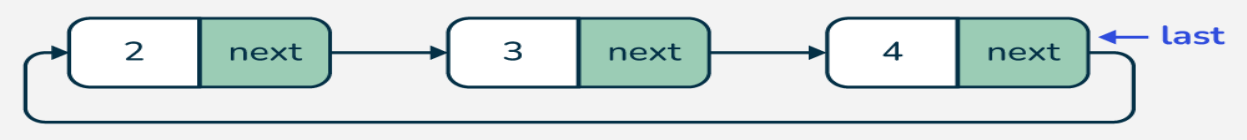


2. Circular Doubly Linked List:

In circular doubly linked list, each node has two pointers prev and next, similar to doubly linked list. The prev pointer points to the previous node and the next points to the next node. Here, in addition to the last node storing the address of the first node, the first node will also store the address of the last node.



Here's an example of creating a circular linked list with three nodes (2, 3, 4):



Operations on the Circular Linked list

We can do some operations on the circular linked list similar to the singly and doubly linked list which are:

1. Insertion

- Insertion at the empty list
- Insertion at the beginning
- Insertion at the end
- Insertion at the given position

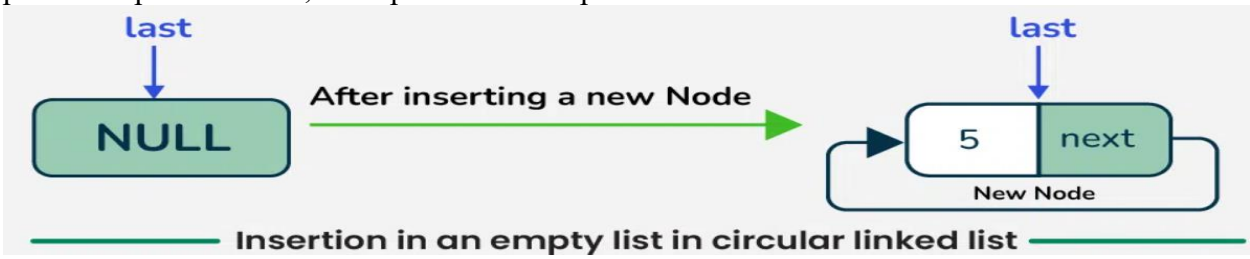
2. Deletion

- Delete the first node
- Delete the last node
- Delete the node from any position

3. Searching

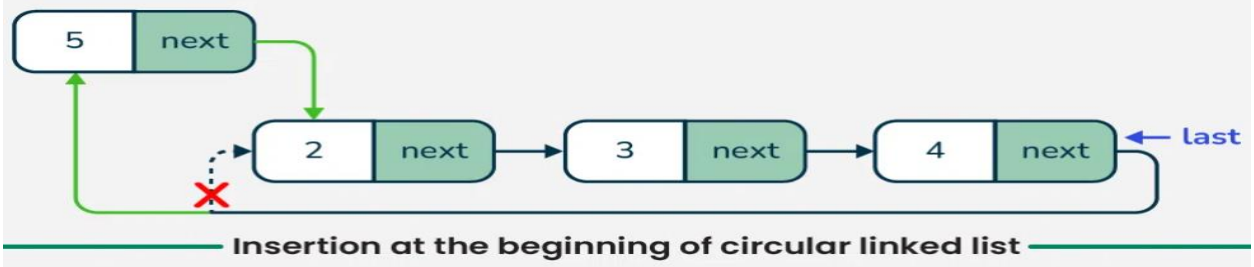
1. Insertion in an empty List in the circular linked list

To insert a node in empty circular linked list, creates a new node with the given data, sets its next pointer to point to itself, and updates the last pointer to reference this new node.



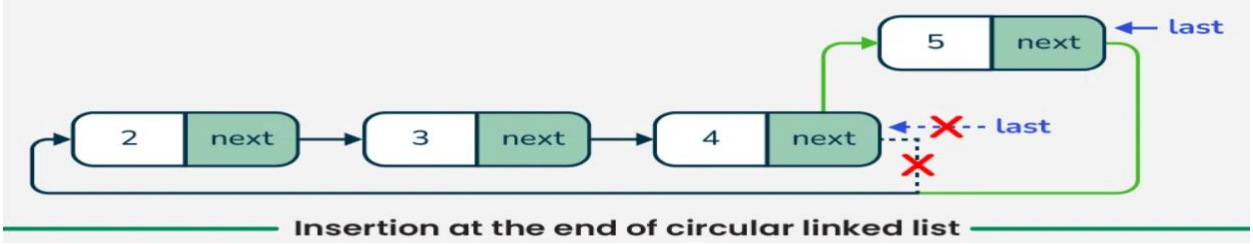
2. Insertion at the beginning in circular linked list

To insert a new node at the beginning of a circular linked list, we create a new node and check if the list is empty. If empty, the new node points to itself. If not, we make the new node's next pointer point to the current head (last->next) and update the last node's next to the new node, preserving the circular structure.



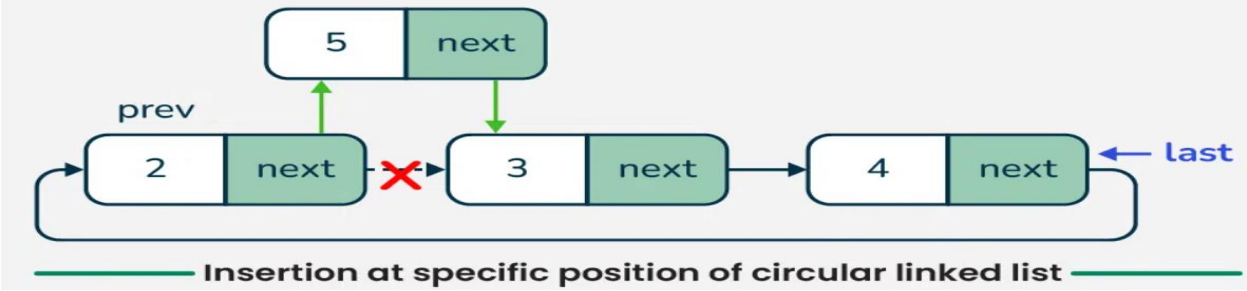
3. Insertion at the end in circular linked list

To insert a node at the end of a circular linked list, we create the new node and, if the list is empty, make it point to itself. Otherwise, we update the tail's next pointer to the new node and then set the tail to the new node, preserving the circular linkage.



4. Insertion at specific position in circular linked list

To insert a node at a specific position in a circular linked list, we handle edge cases for an empty list and invalid positions. For valid positions, we traverse the list and adjust the pointers to insert the new node, updating the tail if it's inserted at the end.



Deletion from a Circular Linked List

Deletion involves removing a node from the linked list. The main difference is that we need to ensure the list remains circular after the deletion. We can delete a node in a circular linked list in three ways:

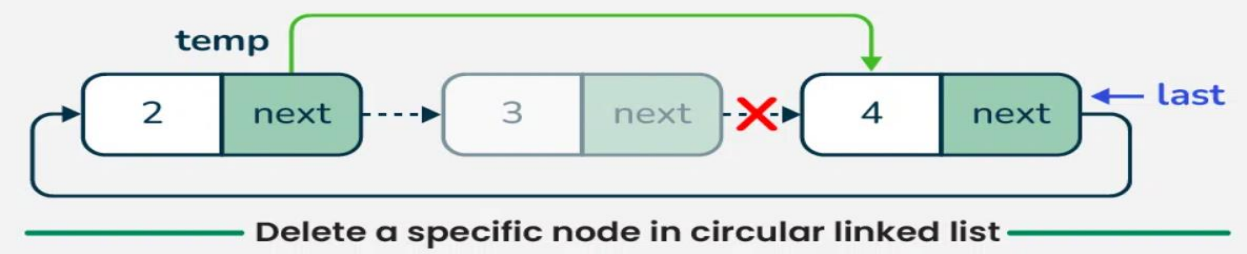
1. Delete the first node in circular linked list

To delete the first node of a circular linked list, we check if the list is empty or has only one node. If so, we handle those cases by deleting the node and updating the last pointer. For multiple nodes, we update the last node's next pointer to skip the head and free the head node, returning the updated last pointer.



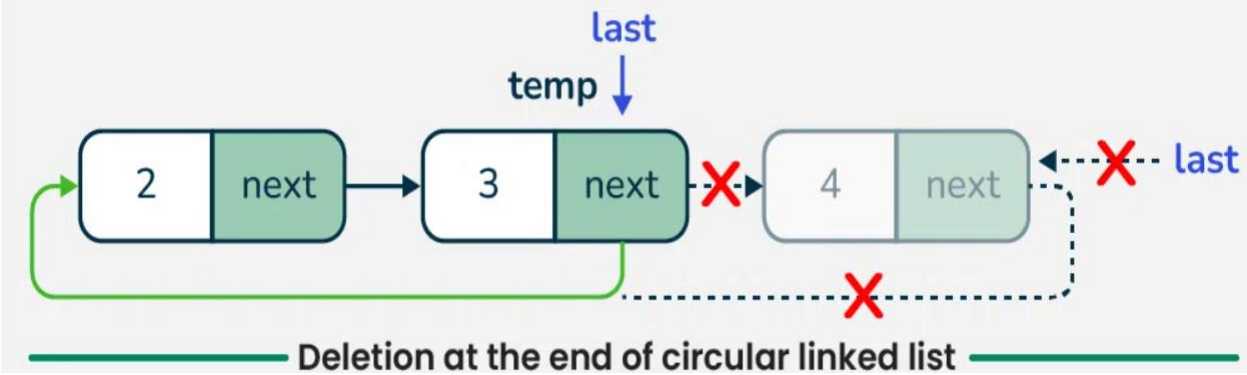
2. Delete a specific node in circular linked list

To delete a specific node from a circular linked list, we handle empty list and single node cases. For other nodes, we use two pointers to find the node, update the previous node's next pointer to skip the target, and delete it, updating the last pointer if needed.



3. Deletion at the end of Circular linked list

To delete the last node in a circular linked list, we handle the empty and single node cases. For multiple nodes, we traverse to find the second last node, update its next pointer to the head, delete the last node, and return the updated last pointer.



Write a C Program to implement circular linked list operations

```
//Circular single linked list
#include<stdio.h>
#include<stdlib.h>
typedef struct node
{
        int data;
        struct node *next;
} NODE;
NODE *head=NULL;
NODE *create();
void insertAtBegin();
void insertAtEnd();
void insertAtAnywhere();
void deleteAtBegin();
void deleteAtEnd();
void deleteAtAnywhere();
void traverse();
int size=0;
NODE *create()
{
        NODE *newNode=(NODE*)malloc(sizeof(NODE));
        printf("\nEnter Data:");
        scanf("%d",&newNode->data);
        size++;
        return newNode;
}
void traverse()
{
        if(head==NULL)
        printf("List is Empty!!!\n");
        else
        {
                NODE *temp=head;
                do
                {
                        printf("-->%d",temp->data);
                        temp=temp->next;
                }while(temp!=head);
                printf("\n");
        }
}
void insertAtBegin()
{
        NODE *newNode=create(),*t;
        if(head==NULL)
```

```
	{
		head=newNode;
		head->next=head;
	}
	else
	{
		t=head;
		while(t->next!=head)
		{
			t=t->next;
		}
		newNode->next=head;
		t->next=newNode;
		head=newNode;
	}
}
void insertAtEnd()
{
	NODE *newNode=create(),*t;
	if(head==NULL)
	{
		head=newNode;
		head->next=head;
	}
	else
	{
		newNode->next=head;
		t=head;
		while(t->next!=head)
		t=t->next;
		t->next=newNode;
	}
}
void insertAtAnywhere()
{
	int p,loc=0;
	printf("Enter position to insert:");
	scanf("%d",&p);
	if(head==NULL)
	printf("No Such postion\n");
	else if(p==1)
	insertAtBegin();
	else if(p==size+1)
	insertAtEnd();
	else if(p>size)
	printf("No Such postion\n");
```

```
        else
        {
                NODE *newNode=create(),*t,*pr;
                t=pr=head;
                while(t->next!=head)
                {
                        loc++;
                        if(loc==p)
                        break;
                        pr=t;
                        t=t->next;
                }
                newNode->next=t;
                pr->next=newNode;
        }
}
void deleteAtBegin()
{
        if(head==NULL)
        printf("List is Empty!!!\n");
        else if(size==1)
        {
                NODE *temp=head;
                printf("%d is removed\n",temp->data);
                size--;
                head=NULL;
                free(temp);
        }
        else
        {
                NODE *t,*temp;
                t=temp=head;
                while(t->next!=head)
                t=t->next;
                t->next=head->next;
                head=head->next;
                printf("%d is removed\n",temp->data);
                size--;
                free(temp);
        }
}
void deleteAtEnd()
{
        NODE *temp,*t;
        temp=t=head;
        if(head==NULL)
```

```
        printf("List is empty!!!\n");
        else if(size==1)
        {
                printf("%d is removed\n",temp->data);
                size--;
                head=NULL;
                free(temp);
        }
        else
        {
                while(temp->next!=head)
                {
                        t=temp;
                        temp=temp->next;
                }
                t->next=head;
                printf("%d is removed\n",temp->data);
                size--;
                free(temp);
        }
}
void deleteAtAnywhere()
{
        NODE *temp,*t;
        temp=t=head;
        if(head==NULL)
        printf("List is empty!!!\n");
        else
        {
                int p,loc=0;
                printf("Enter position to delete:");
                scanf("%d",&p);
                if(p==1)
                deleteAtBegin();
                else if(p==size)
                deleteAtEnd();
                else if(p>size)
                printf("No Such position\n");
                else
                {
                        while(temp->next!=head)
                        {
                                loc++;
                                if(loc==p)
                                break;
                                t=temp;
```

```
                        temp=temp->next;
                }
                t->next=temp->next;
                printf("%d is removed\n",temp->data);
                size--;
                free(temp);
        }
    }
}

int main()
{
    int ch;
    do
    {
        printf("1.Insert at beginning.\n");
        printf("2.Insert At Ending.\n");
        printf("3.Insert At Any where.\n");
        printf("4.Deletion at bigenning.\n");
        printf("5.Deletion at ending.\n");
        printf("6.Deletion At Anywhere.\n");
        printf("7.Traverse.\n");
        printf("8.Count.\n");
        printf("Enter your choice:");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:insertAtBegin();
                   break;
            case 2:insertAtEnd();
                   break;
            case 3:insertAtAnywhere();
                   break;
            case 4:deleteAtBegin();
                   break;
            case 5:deleteAtEnd();
                   break;
            case 6:deleteAtAnywhere();
                   break;
            case 7:traverse();
                   break;
            case 8:printf("Number of elements are %d\n",size);
                   break;
            default:exit(0);
        }
    }while(1);
```

```
    return 0;
}
```

Output:

```
1.Insert at beginning.
2.Insert At Ending.
3.Insert At Any where.
4.Deletion at bigenning.
5.Deletion at ending.
6.Deletion At Anywhere.
7.Traverse.
8.Count.
Enter your choice:1
Enter Data:10
1.Insert at beginning.
2.Insert At Ending.
3.Insert At Any where.
4.Deletion at bigenning.
5.Deletion at ending.
6.Deletion At Anywhere.
7.Traverse.
8.Count.
Enter your choice:2
Enter Data:20
1.Insert at beginning.
2.Insert At Ending.
3.Insert At Any where.
4.Deletion at bigenning.
5.Deletion at ending.
6.Deletion At Anywhere.
7.Traverse.
8.Count.
Enter your choice:1
Enter Data:30
1.Insert at beginning.
2.Insert At Ending.
3.Insert At Any where.
4.Deletion at bigenning.
5.Deletion at ending.
6.Deletion At Anywhere.
7.Traverse.
8.Count.
Enter your choice:2
Enter Data:40
1.Insert at beginning.
2.Insert At Ending.
3.Insert At Any where.
```

```
1.Insert at beginning.
2.Insert At Ending.
3.Insert At Any where.
4.Deletion at bigenning.
5.Deletion at ending.
6.Deletion At Anywhere.
7.Traverse.
8.Count.
Enter your choice:1
Enter Data:60
1.Insert at beginning.
2.Insert At Ending.
3.Insert At Any where.
4.Deletion at bigenning.
5.Deletion at ending.
6.Deletion At Anywhere.
7.Traverse.
8.Count.
Enter your choice:2
Enter Data:80
1.Insert at beginning.
2.Insert At Ending.
3.Insert At Any where.
4.Deletion at bigenning.
5.Deletion at ending.
6.Deletion At Anywhere.
7.Traverse.
8.Count.
Enter your choice:3
Enter position to insert:4
Enter Data:70
1.Insert at beginning.
2.Insert At Ending.
3.Insert At Any where.
4.Deletion at bigenning.
5.Deletion at ending.
6.Deletion At Anywhere.
7.Traverse.
8.Count.
Enter your choice:7
-->60-->30-->10-->70-->50-->20-->40-->80
1.Insert at beginning.
```

```
1.Insert at beginning.
2.Insert At Ending.
3.Insert At Any where.
4.Deletion at bigenning.
5.Deletion at ending.
6.Deletion At Anywhere.
7.Traverse.
8.Count.
Enter your choice:4
90 is removed
1.Insert at beginning.
2.Insert At Ending.
3.Insert At Any where.
4.Deletion at bigenning.
5.Deletion at ending.
6.Deletion At Anywhere.
7.Traverse.
8.Count.
Enter your choice:5
80 is removed
1.Insert at beginning.
2.Insert At Ending.
3.Insert At Any where.
4.Deletion at bigenning.
5.Deletion at ending.
6.Deletion At Anywhere.
7.Traverse.
8.Count.
Enter your choice:7
-->60-->30-->10-->70-->50-->20-->40
1.Insert at beginning.
2.Insert At Ending.
3.Insert At Any where.
4.Deletion at bigenning.
5.Deletion at ending.
6.Deletion At Anywhere.
7.Traverse.
8.Count.
Enter your choice:3
Enter position to insert:9
No Such postion
1.Insert at beginning.
```

4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Count. Enter your choice:3 Enter position to insert:3 Enter Data:50 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Count. Enter your choice:7 -->30-->10-->50-->20-->40 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Count. Enter your choice:8 Number of elements are 5	2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Count. Enter your choice:1 Enter Data:90 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Count. Enter your choice:7 -->90-->60-->30-->10-->70-->50-->20-->40-->80 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Count. Enter your choice:8 Number of elements are 9	2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Count. Enter your choice:6 Enter position to delete:3 10 is removed 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Count. Enter your choice:7 -->60-->30-->70-->50-->20-->40 1.Insert at beginning. 2.Insert At Ending. 3.Insert At Any where. 4.Deletion at bigenning. 5.Deletion at ending. 6.Deletion At Anywhere. 7.Traverse. 8.Count. Enter your choice:10

Compare linked list and array:

Feature	Linked List	Array
Data Structure Type	Linear data structure with nodes pointing to the next node	Linear data structure with contiguous memory locations
Memory Allocation	Dynamic (allocated during runtime)	Static or dynamic (depending on the language)
Access Time	Sequential access (O(n))	Random access (O(1))
Insertion/Deletion	Efficient (O(1) if the pointer is given)	Inefficient (O(n) due to shifting)
Memory Usage	Extra memory is required for pointers	Memory-efficient, no extra pointer required
Flexibility	Can easily grow and shrink in size	Fixed size (if static) or resizing can be costly

Cache Locality	Poor (nodes are scattered in memory)	Excellent (contiguous memory locations)
Reverse Traversal	Difficult (without a doubly linked list)	Easy with index manipulation
Data Storage	Can store different data types (if allowed by language)	Typically homogeneous data
Implementation Complexity	Complex due to pointer management	Simple and straightforward

Applications of Linked List:

Applications of linked list in computer science:

1. Implementation of stacks and queues
2. Implementation of graphs: Adjacency list representation of graphs is the most popular which uses a linked list to store adjacent vertices.
3. Dynamic memory allocation: We use a linked list of free blocks.
4. Maintaining a directory of names
5. Performing arithmetic operations on long integers
6. Manipulation of polynomials by storing constants in the node of the linked list
7. Representing sparse matrices
8. Redo and undo functionality.
9. Use of the Back and forward button in a browser.
10. The most recently used section is represented by the Doubly Linked list.
11. Other Data structures like Stack, Hash Table, and Binary Tree can also be applied by Doubly Linked List.
12. Used to implement game objects and their interactions in a game engine.
13. Used in networking.
14. Used in Graph algorithms.
15. Operating systems use doubly linked lists to manage the process scheduling. Each process waiting to be executed is represented as a node in the doubly linked list, and the operating system can easily traverse the list in both directions to manage the process queue.